A

Report

on

DEVOPS MINI PROJECT

PG EXPENSE SPLITTER

Implementation of CI/CD, Docker Containerization and Application Monitoring

Submitted in partial fulfillment of the requirements for the

degree

Third Year Engineering – Computer Science Engineering (Data Science)

By

Charukesh Addugula (24107139)

Atharva Late (24107059)

Kunal Kudu (24107044)

Under the guidance of

Prof. shubhangi soni



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2026-27

Approval Sheet

This Mini Project Report entitled **"PG Expense Splitter – DevOps Mini Project"**, submitted by **Charukesh Addugula (24107139), Atharva Late (24107059), and Kunal Kudu (24107044)**, is presented in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science Engineering (Data Science) from the University of Mumbai.

The project focuses on the application of DevOps practices, including version control using Git and GitHub, CI pipeline configuration, Docker containerization, deployment, and application monitoring using Prometheus and Grafana.

Mini project Guide

Prof. shubhangi soni

ABSTRACT

The PG Expense Splitter is a web-based application developed to simplify the management of shared expenses among students living in Paying Guest (PG) accommodations. It allows users to join groups using a unique PG code, record shared expenses, calculate individual contributions, and track payment status. The application uses HTML, CSS, and JavaScript for the frontend, PHP for backend processing, and MySQL for data storage.

This DevOps mini project focuses on applying DevOps practices to the existing PG Expense Splitter application. Git and GitHub are used for version control and source code management. GitHub Actions is used to configure the Continuous Integration (CI) workflow. Docker is used to containerize the application and database services, helping provide a consistent execution environment.

Prometheus and Grafana are used for monitoring and visualizing system metrics. These tools help observe the monitoring targets and present available metrics through dashboards. Screenshots of the application, GitHub repository, CI workflow, Docker containers, and monitoring setup are included as implementation evidence.

The project demonstrates how an existing web application can be supported by version control, automated workflows, containerization, deployment configuration, and monitoring tools. These practices improve consistency, simplify application management, and provide a foundation for future improvements in reliability and maintainability.

CONTENTS

Chapter No.	Chapter Title
1	Introduction
2	Tools and Technologies Used
3	System and DevOps Architecture
4	Application Implementation
5	Git and GitHub Implementation
6	CI Pipeline Using GitHub Actions
7	Dockerization and Deployment
8	Testing and Results
9	Monitoring Using Prometheus and Grafana
10	Advantages, Applications and Limitations
11	Conclusion and Future Scope
12	References

Chapter 1: Introduction

1.1 Background

In modern urban living, many individuals reside in PG accommodations where expenses are shared among multiple members. Managing these shared expenses manually often leads to confusion, disputes, and lack of transparency.

1.2 Problem Statement

Traditional methods of expense management (manual calculation, WhatsApp messages, spreadsheets) are inefficient and error-prone. There is a need for a system that can:

a) Automatically split expenses.

b) Track payments.

c) Maintain transparency among users.

1.3 Objective

The main objective of this project is to develop a system that:

a) Allows users to join a PG group using a unique code.

b) Enables admins to manage and split expenses.

c) Tracks payment status (Paid/Pending).

d) Provides a clear view of expense distribution.

1.4 Scope

The system is designed for:

a) PG accommodations.

b) Shared flats or hostels.

c) Small groups managing shared expenses.

1.5 DevOps Project Overview

The existing PG Expense Splitter application is used to manage and split shared expenses among members of a Paying Guest accommodation. In this mini project, DevOps practices are applied to support the application's source code management, integration, containerization, deployment, and monitoring.

Git and GitHub are used to maintain and manage the project source code. GitHub Actions is used to define a CI workflow. Docker is used to package the application and its required services into containers. Prometheus and Grafana are used to monitor available system metrics and display them through dashboards.

The purpose of this DevOps implementation is to establish a more organized development and deployment workflow while maintaining the existing functionality of the PG Expense Splitter application.

1.6 DevOps Objectives

1. To maintain project source code using Git and GitHub.
2. To configure a Continuous Integration workflow using GitHub Actions.
3. To containerize the application and required services using Docker.
4. To configure the application execution environment.
5. To monitor available system metrics using Prometheus and Grafana.
6. To document the implementation process with screenshots and test results.

Chapter 2: Tools, Technologies Used

2.1 Frontend Technologies

HTML: Used to structure the web pages of the PG Expense Splitter application.

CSS: Used to style the interface and improve its visual appearance.

JavaScript: Used to provide client-side interactivity.

2.2 Backend Technology

PHP: Used to handle application logic, user authentication, expense processing, and communication with the database.

2.3 Database Technology

MySQL: Used to store user information, group details, and expense records. The database supports the application's data management requirements.

2.4 Development Tools

Visual Studio Code: Used as the source code editor.

XAMPP: Used as the local development environment for running the PHP application and MySQL database during development.

2.5 Version Control Tools

Git: Used to track source code changes and maintain project versions.

GitHub: Used to host the project repository, collaborate on source code, and maintain the GitHub Actions workflow configuration.

2.6 Continuous Integration Tool

GitHub Actions: Used to define an automated workflow for the project. The workflow configuration is stored in the repository under `.github/workflows/`. The configured workflow can perform the steps specified in its YAML file, such as building or testing the application.

Jenkins: A CI/CD automation server included in the project tool selection. Its actual configuration and execution results should be documented only if implemented and verified.

2.7 Containerization Tool

Docker: Used to package the application and its dependencies into containers, helping maintain a consistent execution environment.

Docker Compose: Used to define and manage multiple related services, such as the web application and database, through a Compose configuration file.

2.8 Monitoring Tools

Prometheus: Used to collect and monitor metrics from configured monitoring targets.

Grafana: Used to visualize available metrics through dashboards, helping users observe system performance and monitoring status.

2.9 Summary

The project combines web development technologies with DevOps tools for source code management, CI workflow configuration, containerization, and monitoring. These tools support an organized application lifecycle from development to execution and observation.

Chapter 3: System And Devops Architecture

3.1 System Overview

The PG Expense Splitter is a web-based application designed to manage shared expenses among members of a Paying Guest (PG) accommodation. The application allows users to register, log in, join a group using a unique PG code, and view shared expenses. Administrators can manage group expenses and track payment status.

The application uses HTML, CSS, and JavaScript for the frontend, PHP for backend processing, and MySQL for storing application data. Docker is used to run the application and its database services in containers.

3.2 System Architecture

The system follows a client-server architecture. Users interact with the application through a web browser. The frontend sends requests to the PHP backend, which processes the requests and communicates with the MySQL database. The processed information is returned to the user through the web interface.

The major components are:

1. **Client:** A web browser used to access the PG Expense Splitter.
2. **Frontend:** HTML, CSS, and JavaScript provide the user interface.
3. **Backend:** PHP handles authentication, expense management, and application logic.
4. **Database:** MySQL stores users, groups, and expense records.
5. **Container Environment:** Docker containers provide an environment for running the application and database services.

3.3 DevOps Architecture

The DevOps workflow connects source code management, continuous integration, containerization, application execution, and monitoring.

Git is used to track code changes, while GitHub hosts the project repository. GitHub Actions provides a workflow configuration for the CI process. Docker and Docker Compose are used to build and manage

the application environment. Prometheus monitors configured targets, and Grafana displays available metrics through dashboards.

Jenkins is considered part of the planned DevOps toolset; its integration should be documented as implemented only after its configuration and execution have been verified.

3.4 DevOps Workflow

The project's DevOps workflow consists of the following stages:

1. **Development:** Application source code is created and updated.
2. **Version Control:** Git tracks code changes.
3. **Code Hosting:** GitHub stores the source code and workflow configuration.
4. **Continuous Integration:** GitHub Actions executes the steps defined in the workflow file.
5. **Containerization:** Docker packages the application and related services into containers.
6. **Execution:** Docker Compose can be used to manage the configured application services.
7. **Monitoring:** Prometheus collects metrics from configured targets, and Grafana presents available monitoring information.

3.5 Benefits of the Architecture

1. Separates frontend, backend, and database responsibilities.
2. Maintains source code history using version control.
3. Supports repeatable application environments through containerization.
4. Provides a structured workflow for integration and testing.
5. Improves visibility into configured monitoring targets and system metrics.

3.6 Summary

The system architecture describes how users interact with the PG Expense Splitter application, while the DevOps architecture describes how the source code is managed, integrated, containerized, executed, and monitored. Together, these components provide a structured approach to application development and operations.

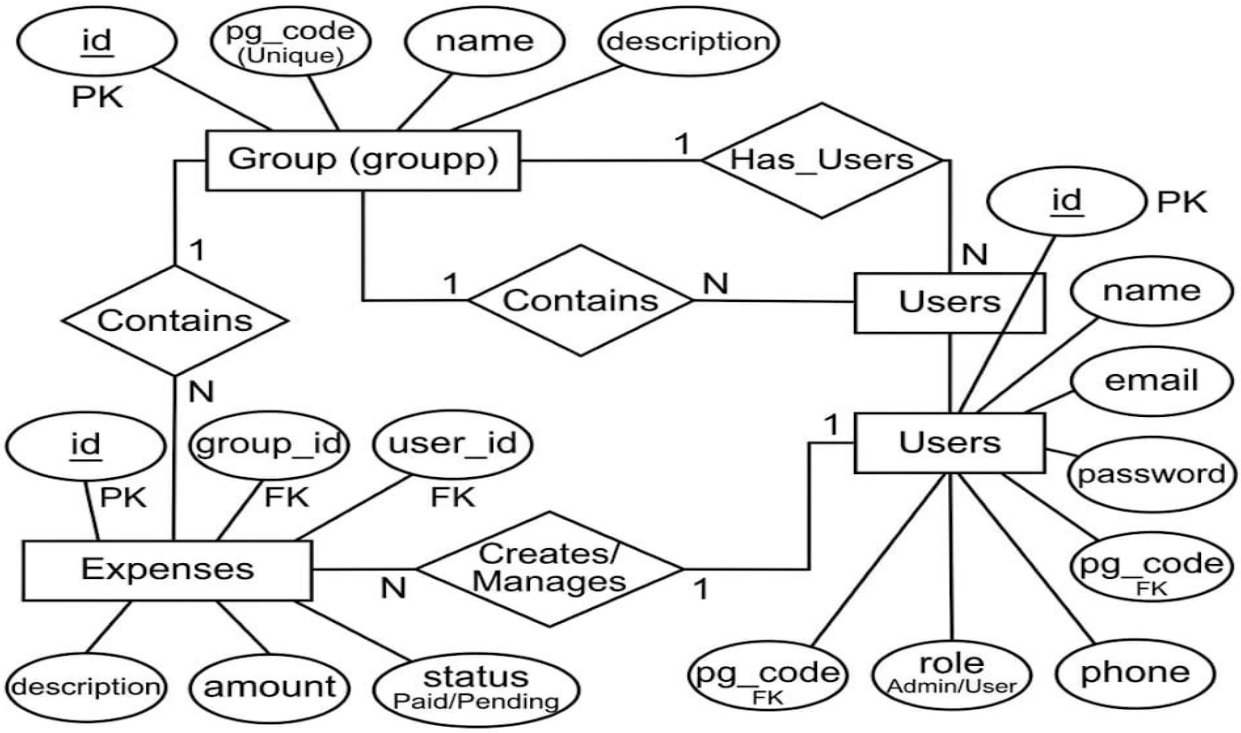


Figure 3.1

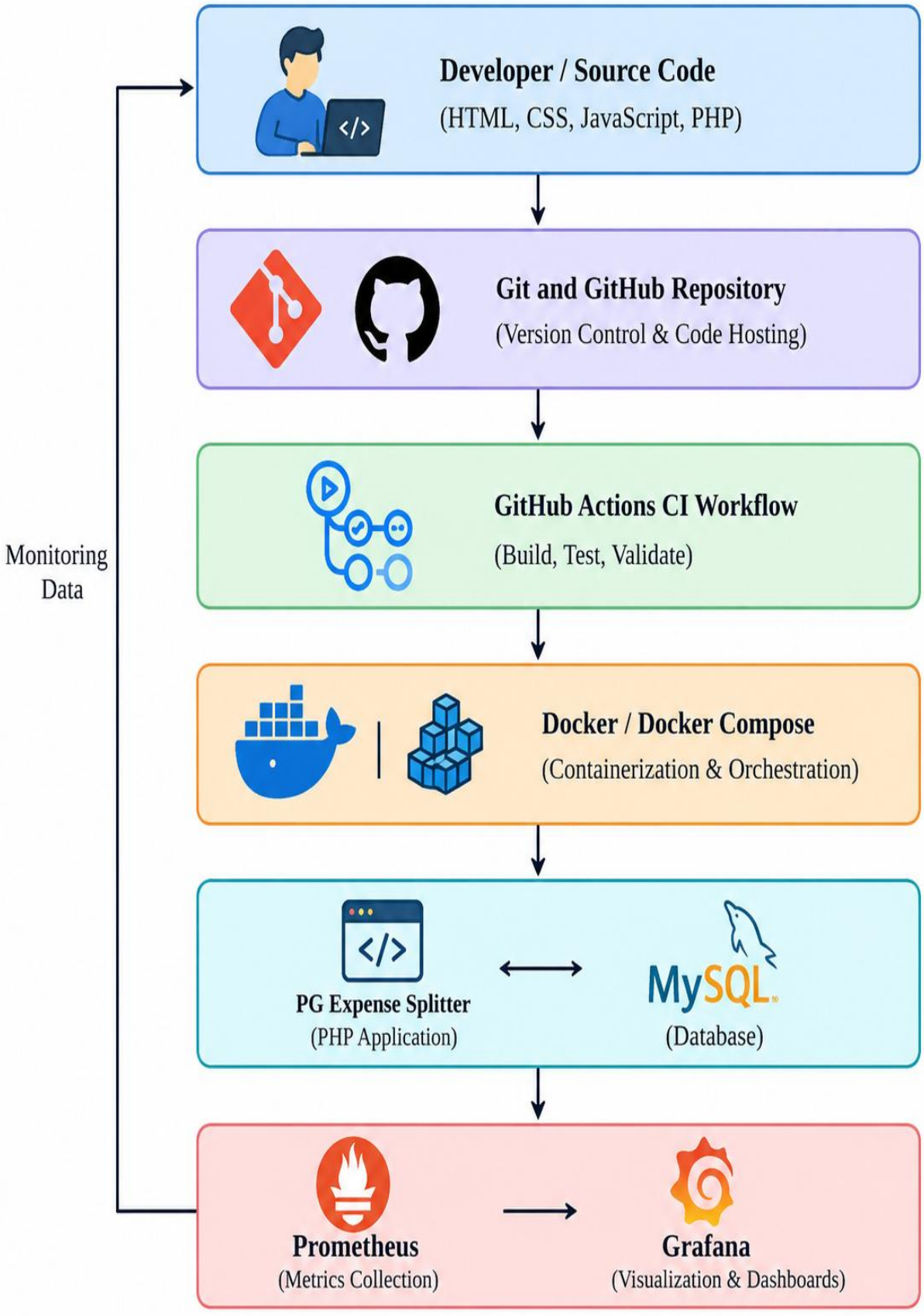

Developer / Source Code
(HTML, CSS, JavaScript, PHP)
Git and GitHub Repository
(Version Control & Code Hosting)
GitHub Actions CI Workflow
(Build, Test, Validate)
Monitoring Data
Docker / Docker Compose
(Containerization & Orchestration)
PG Expense Splitter
(PHP Application)
MySQL
(Database)
Prometheus
(Metrics Collection)
Grafana
(Visualization & Dashboards)

Figure 3.2

Chapter 4: Implementation Details

4.1 Database Design

The database consists of three main tables:

1. groupp
2. users
3. expenses

Foreign key constraints ensure data integrity:

users.pg_code → groupp.pg_code

expenses.group_id → groupp.id

expenses.user_id → users.id

```
PS C:\xampp\htdocs\pg-expense-splitter-devops> docker compose exec db maria
db -u pgapp -ppgapp_local pg_expenses -e "SHOW TABLES;"
+----------------------+
| Tables_in_pg_expenses |
+----------------------+
| expenses             |
| groupp               |
| users                |
+----------------------+
PS C:\xampp\htdocs\pg-expense-splitter-devops>
```

Figure 4.1: MySQL Database Tables

4.2 Authentication System

1. Users login using email and password.
2. Admin has additional privileges.

3. Role-based access control is implemented.

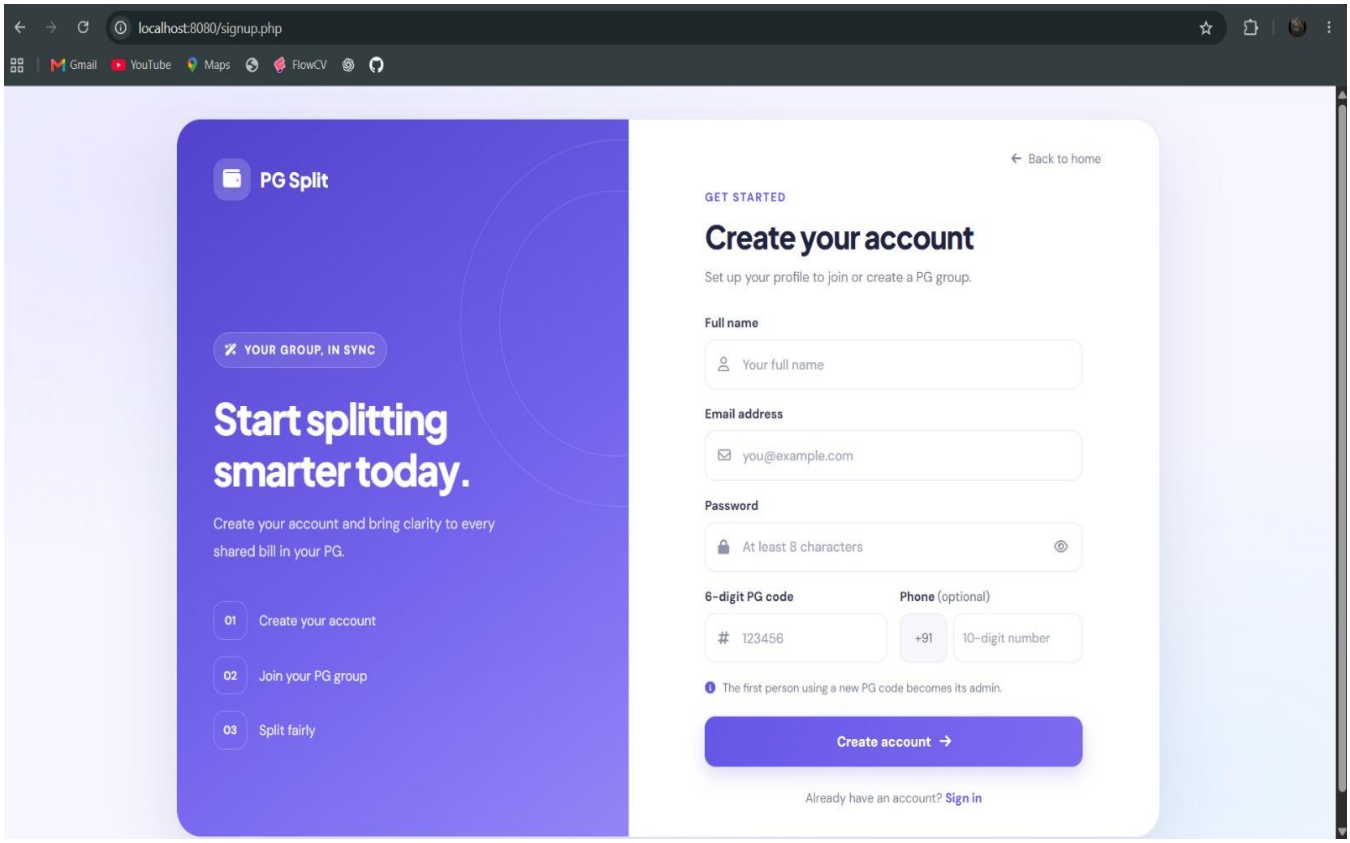


Figure 4.2: User Signup Interface

4.3 Group Management

1. Admin creates a group with a unique PG code.
2. Users join using the PG code.

4.4 Expense Management

1. Admin enters expense details (description, amount).
2. System splits the amount among users.
3. Each user gets their share.

4.5 User Interface Features

1. Dashboard with group info.
2. Expense table.
3. Split calculator.
4. Clean navigation (Welcome, Group, Logout).

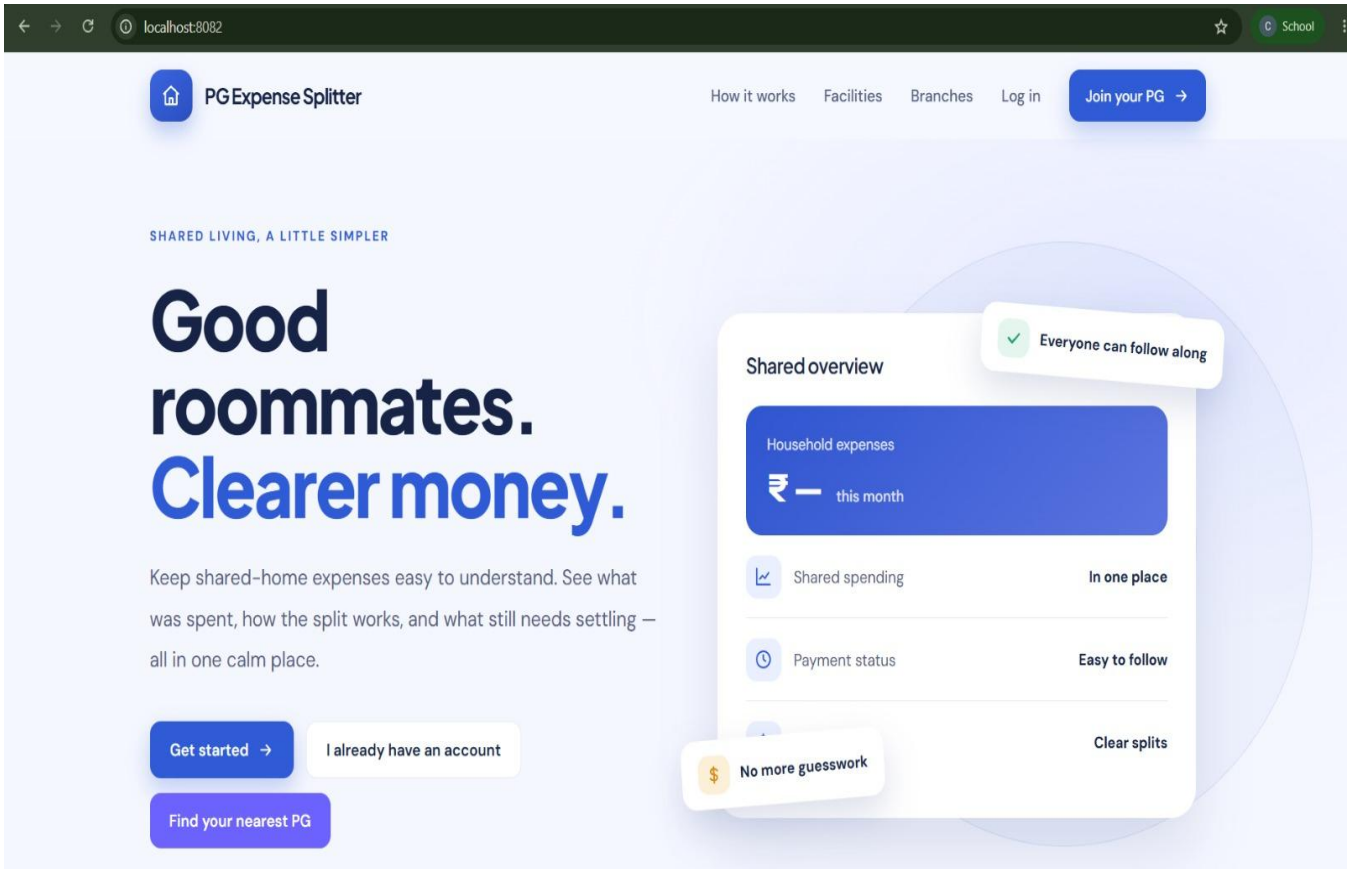


Figure 4.3: PG Expense Splitter Homepage

4.6 MySQL Code :

```sql
-- Drop old database if it exists
DROP DATABASE IF EXISTS pg_expenses;

-- Create fresh database
CREATE DATABASE pg_expenses;

-- Use the new database
USE pg_expenses;

CREATE TABLE groupp (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pg_code VARCHAR(6) UNIQUE,
    name VARCHAR(100),
    description VARCHAR(255)
);
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(255),
    pg_code VARCHAR(6),
    role ENUM('Owner','Admin','User') DEFAULT 'User',
    phone VARCHAR(15),
    FOREIGN KEY (pg_code) REFERENCES groupp(pg_code) ON DELETE CASCADE
);


CREATE TABLE expenses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT,
    user_id INT,
    description VARCHAR(255),
    amount DECIMAL(10,2),
    status ENUM('Paid','Pending') DEFAULT 'Pending',
    FOREIGN KEY (group_id) REFERENCES groupp(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Figure 4.4 Database Design

Chapter 5: Git And Github Implementation

5.1 Introduction

Git and GitHub are important tools for source code management in the DevOps lifecycle. Git is a version control system that tracks changes made to project files, while GitHub provides an online platform to host and manage the project repository.

In the PG Expense Splitter project, these tools support source code organization and provide a foundation for continuous integration and containerization.

5.2 Git Implementation

Git helps maintain a history of changes made to the application's source code. It allows developers to review modifications and maintain different versions of the project.

The common Git commands used in a project workflow are:

```
git status
git add .
git commit -m "Update project files"
git push
```

Description of commands:

1. `git status` checks the current state of the working directory.
2. `git add .` stages modified and new files.
3. `git commit` records the staged changes.
4. `git push` uploads committed changes to the remote repository.

These commands represent a typical Git workflow. The actual commands used in the project should be confirmed from the repository history.

5.3 GitHub Repository

GitHub is used to host the project's source code online. A repository provides a centralized location for application files and configuration files.

The PG Expense Splitter repository can contain the frontend files, PHP backend code, database-related files, Docker configuration, and CI workflow files, depending on the implemented project structure.

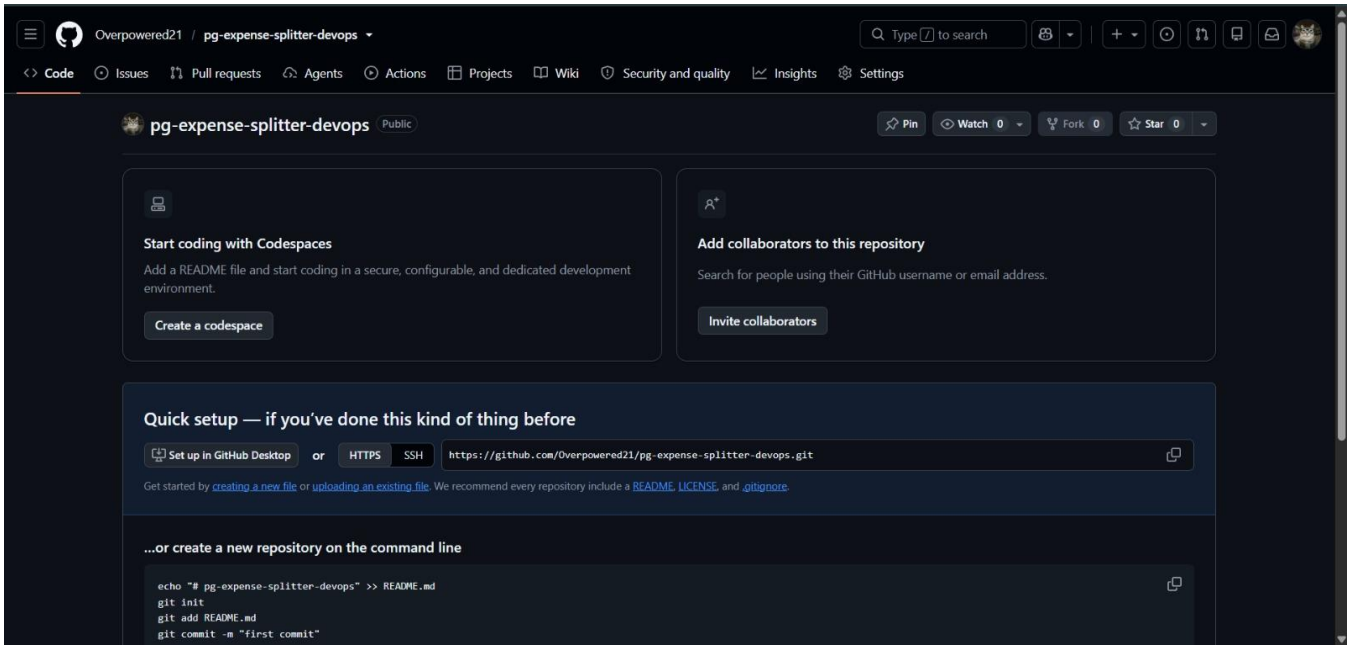


Figure 5.1: GitHub Repository of PG Expense Splitter

5.4 Repository Structure

A typical repository structure for the project may include:

- Application source files
- PHP backend files
- Frontend HTML, CSS, and JavaScript files
- Database configuration or SQL files
- Dockerfile
- Docker Compose configuration
- `.github/workflows/` directory for GitHub Actions workflow files

The exact structure should match the files present in the actual GitHub repository.

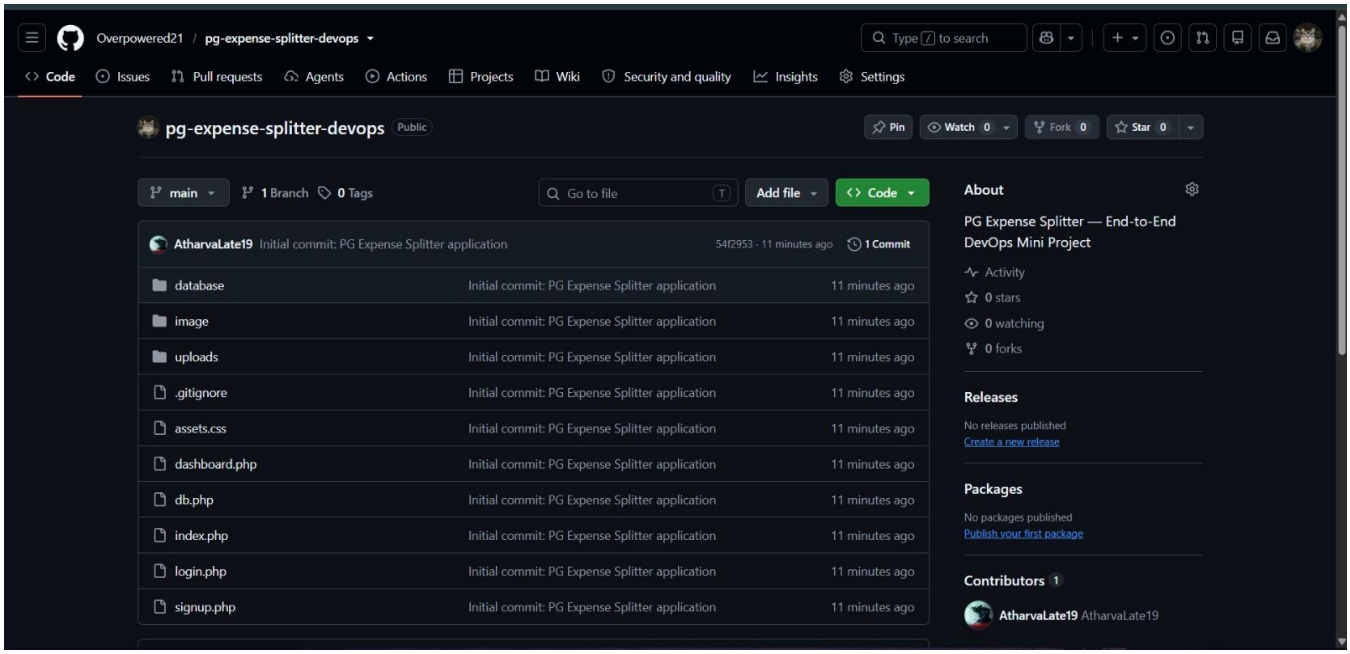


Figure 5.2: Project Files in GitHub Repository

5.5 Role of GitHub in DevOps

GitHub supports the DevOps workflow by storing application source code and workflow configuration. Changes pushed to the repository can trigger automated workflows when the configured conditions are satisfied.

The repository also provides a place to review source code changes and maintain project history.

5.6 Advantages

1. Tracks changes to project files.
2. Maintains source code history.
3. Provides remote repository hosting.
4. Supports collaboration and code review.
5. Integrates with GitHub Actions for workflow automation.

5.7 Summary

Git and GitHub provide the version control and repository management foundation for the PG Expense Splitter DevOps project. The repository also supports integration with CI workflows and containerization configuration, as implemented in the following stages

Chapter 6 : Ci Pipeline Using Github Actions

6.1 Introduction

Continuous Integration (CI) is a DevOps practice in which developers frequently integrate code changes into a shared repository. Automated workflows help verify the application and identify errors during development. In the PG Expense Splitter project, GitHub Actions is used to configure and execute the CI workflow.

6.2 GitHub Actions

GitHub Actions is an automation platform integrated with GitHub. It allows developers to create workflows for building, testing, and validating applications whenever specified repository events occur.

The workflow is defined in a YAML configuration file stored inside the repository. It specifies the trigger events, execution environment, and steps to be performed.

```
PS E:\Softwares\XAMPPPPPP\htdocs\pg-expense-splitter-redesigned> git commit -m "Initial commit: PG Expense Splitter application"
[main (root-commit) 54f2953] Initial commit: PG Expense Splitter application
 10 files changed, 1715 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 assets.css
 create mode 100644 dashboard.php
 create mode 100644 database/schema.sql
 create mode 100644 db.php
 create mode 100644 image/login.webp
 create mode 100644 index.php
 create mode 100644 login.php
 create mode 100644 signup.php
 create mode 100644 uploads/.gitkeep
PS E:\Softwares\XAMPPPPPP\htdocs\pg-expense-splitter-redesigned>
```

Figure 6.1: GitHub Actions Workflow Configuration

6.3 CI Pipeline Workflow

The CI pipeline follows these steps:

1. The developer makes changes to the project source code.
2. The changes are committed and pushed to the GitHub repository.
3. A configured GitHub Actions workflow is triggered.
4. The workflow checks out the repository code.
5. The configured build or validation steps are executed.
6. The workflow displays the execution status as successful or failed.

The exact steps depend on the workflow configuration used in the project.

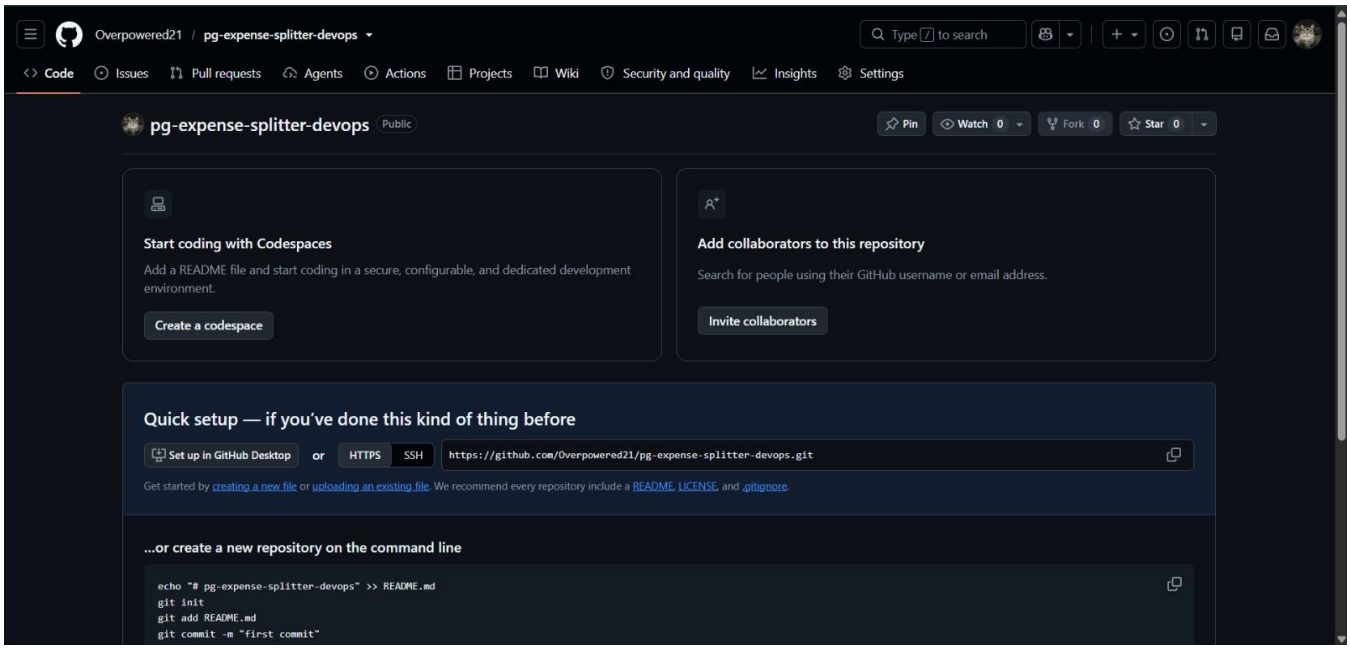


Figure 6.2: GitHub Actions CI Pipeline Execution

6.4 Workflow Configuration

GitHub Actions workflows are stored in the `.github/workflows/` directory. A YAML file defines the trigger conditions, runner environment, and commands.

The workflow can be configured to execute when code is pushed to the repository or when a pull request is created. The required commands should match the technologies and scripts available in the project.

6.5 Benefits of CI

- Automates repetitive development tasks.
- Helps identify errors early.
- Provides a record of workflow executions.
- Improves consistency in code validation.
- Supports collaboration through GitHub.
- Provides a foundation for integrating further deployment automation.

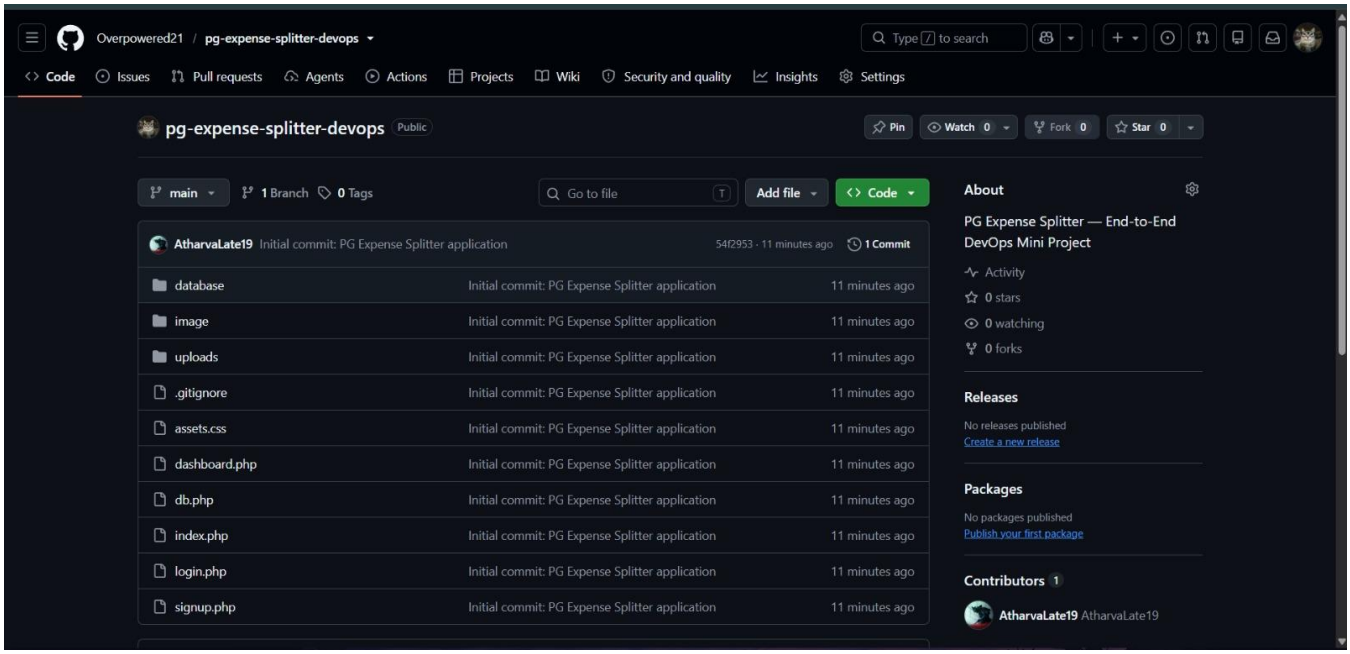


Figure 6.3: GitHub Repository Workflow Files

6.6 Summary

GitHub Actions provides an automated workflow for the PG Expense Splitter project. It connects repository changes with configured validation steps and displays the execution results. This supports the DevOps objective of making software development more consistent and efficient.

Chapter 7: Dockerization And Deployment

7.1 Introduction

Dockerization is the process of packaging an application and its dependencies into containers. Containers provide a consistent environment for running applications and help simplify deployment. In the PG Expense Splitter project, Docker and Docker Compose are used to support application execution and database services.

7.2 Docker

Docker is a containerization platform that allows applications to run in isolated environments. It packages the required application components and dependencies, helping maintain consistency across different systems.

For the PG Expense Splitter project, Docker helps run the application and its supporting services in containers.

7.3 Docker Compose

Docker Compose is used to define and manage multiple containers through a configuration file. It allows related services, such as the application and MySQL database, to be configured and started together.

The project uses Docker Compose to manage the application and database services. The database service status can be checked to verify that it is running correctly.

7.4 Containerization Process

The containerization process involves the following steps:

1. Prepare the application source code and configuration.
2. Define the required services and settings.
3. Configure the application and database containers.
4. Start the services using Docker Compose.
5. Check the status of the running containers.

6. Verify that the database service is healthy and the application is accessible.

```
PS E:\Softwares\XAMPPPPPP\htdocs\pg-expense-splitter-redesigned> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (15/15), 83.31 KiB | 4.17 MiB/s, done.
Total 15 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Overpowered21/pg-expense-splitter-devops.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS E:\Softwares\XAMPPPPPP\htdocs\pg-expense-splitter-redesigned>
```

Figure 7.1: Docker Compose Build and Service Setup

7.5 Deployment Environment

The application is executed in a local development environment using Docker and Docker Compose. The application and MySQL database run as separate services, allowing them to be managed independently.

Container status and database health can be checked using Docker commands. This setup demonstrates containerized execution; it should not be described as cloud deployment unless the application has actually been deployed to a cloud platform.

Live Deployment Link: PG Expense Splitter

https://pg-expense-splitter-devops.onrender.com

The PG Expense Splitter application is deployed on Render and can be accessed through the above URL when the service is available.

```
PS C:\xampp\htdocs\pg-expense-splitter-redesigned> docker compose ps
NAME                                    IMAGE
 COMMAND                  SERVICE   CREATED         STATUS
  PORTS
pg-expense-splitter-redesigned-app-1   pg-expense-splitter-redesigned-app
 "docker-php-entrypoi…"   app       2 minutes ago   Up 2 minutes
  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
pg-expense-splitter-redesigned-db-1    mariadb:10.11
 "docker-entrypoint.s…"   db        2 minutes ago   Up 2 minutes (healthy)
  0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp
PS C:\xampp\htdocs\pg-expense-splitter-redesigned>
```

Figure 7.2: Running Application and Database Containers

7.6 Benefits of Dockerization

- Provides an isolated execution environment.
- Simplifies management of application services.
- Supports consistent application setup.
- Allows multiple services to run together using Docker Compose.
- Makes it easier to start, stop, and inspect containers.
- Supports future deployment to other environments.

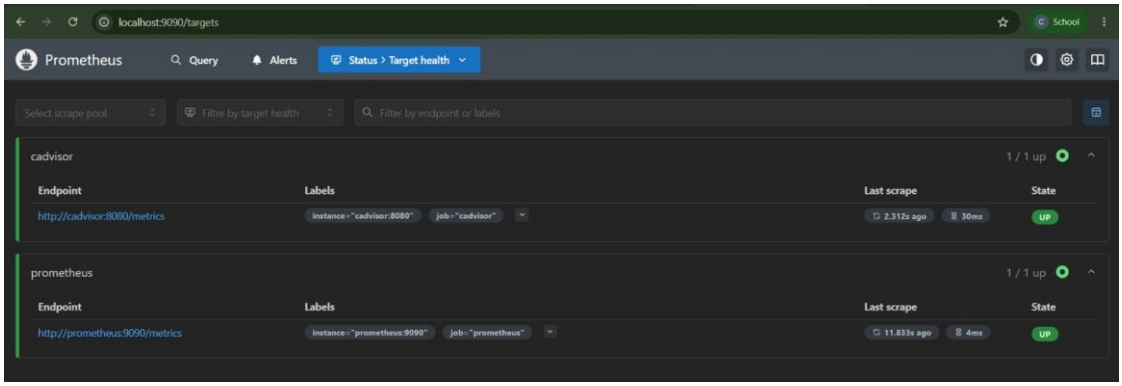


Figure 7.3: Docker Desktop Container Status

7.7 Summary

Docker and Docker Compose help manage the PG Expense Splitter application and its MySQL database in a containerized environment. This approach simplifies service management and provides a foundation for further deployment automation within the DevOps workflow.

Chapter 8 : Testing And Results

8.1 Introduction

Testing is an important phase of software development used to verify whether an application works according to its requirements. The PG Expense Splitter project is tested to check its user interface, authentication, group management, expense management, database connectivity, and container execution.

8.2 Application Testing

The application is tested by checking its major functions. The following test cases describe the expected behaviour of the system.

Sr. No.	Test Case	Expected Result
1	Open the application homepage	Homepage should be displayed
2	Register or log in as a user	Authentication should be processed
3	Access group management	Group-related functions should be available according to user permissions
4	Add or manage an expense	Expense information should be processed and stored
5	Check database tables	Required tables should be present in MySQL
6	Start Docker Compose services	Configured containers should start
7	Check database container health	Database health status should be displayed

The final status of each test case should be recorded after performing the test in the actual application.

```
PS C:\xampp\htdocs\pg-expense-splitter-devops> docker compose exec db maria
db -u pgapp -ppgapp_local pg_expenses -e "SHOW TABLES;"
+----------------------+
| Tables_in_pg_expenses |
+----------------------+
| expenses             |
| groupp               |
| users                |
+----------------------+
PS C:\xampp\htdocs\pg-expense-splitter-devops>
```

Figure 8.1: Verification of MySQL Database Tables

8.3 DevOps Testing

The DevOps setup is verified using the configured tools:

- **Git and GitHub:** Used to maintain the project source code and repository history.
- **GitHub Actions:** Used to execute the configured CI workflow and display its status.
- **Docker and Docker Compose:** Used to run and inspect the application and database containers.
- **Prometheus:** Used to check the status of configured monitoring targets.
- **Grafana:** Used to display monitoring information through dashboards.

Only the checks actually performed and their observed results should be reported.

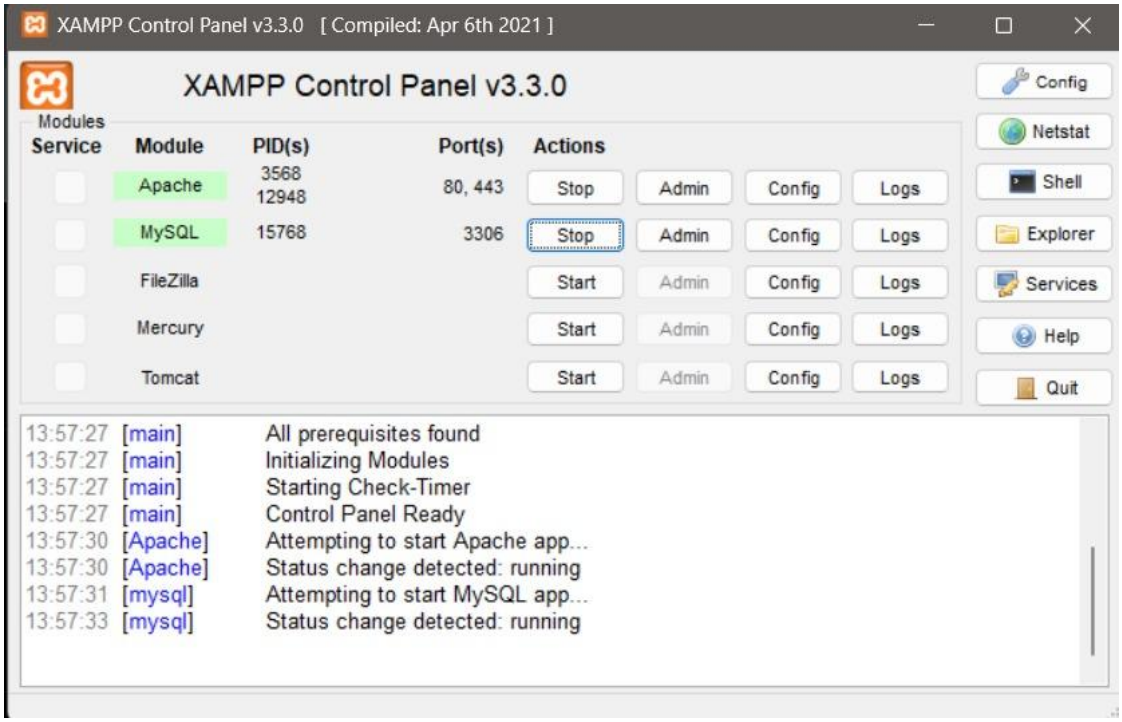


Figure 8.2: Local Development Services in XAMPP

8.4 Results and Output Analysis

The project provides a web interface for managing PG-related group expenses. The database contains tables for users, groups, and expenses. The Docker environment supports running the application and database as separate services.

The GitHub repository provides source-code management, while GitHub Actions provides visibility into configured workflow executions. Prometheus and Grafana support monitoring of the configured targets and visualisation of available metrics.

8.5 Conclusion

The testing process helps verify the major application functions and the configured DevOps environment. The project demonstrates the use of source-code management, CI workflow automation, containerization, and monitoring tools in the development of the PG Expense Splitter application.

Chapter 9 :Monitoring Using Prometheus And Grafana

9.1 Introduction

Monitoring is an important part of DevOps because it helps observe the status and performance of systems and services. Prometheus and Grafana are monitoring tools used to collect, inspect, and visualise system metrics. In the PG Expense Splitter project, these tools are used to monitor the configured targets and display available metrics.

9.2 Prometheus

Prometheus is an open-source monitoring and alerting toolkit. It collects metrics from configured targets and stores them as time-series data. Its web interface allows users to inspect monitoring targets and check their status.

In this project, the Prometheus Targets page is used to verify the status of the configured monitoring targets. A target displayed as UP indicates that Prometheus can successfully scrape metrics from that target.

9.3 Grafana

Grafana is a visualisation and monitoring platform used to display metrics through dashboards. It can connect to supported data sources, including Prometheus, and present the collected information using charts and panels.

In this project, Grafana is used to view the configured monitoring dashboard and visualise the available metrics.

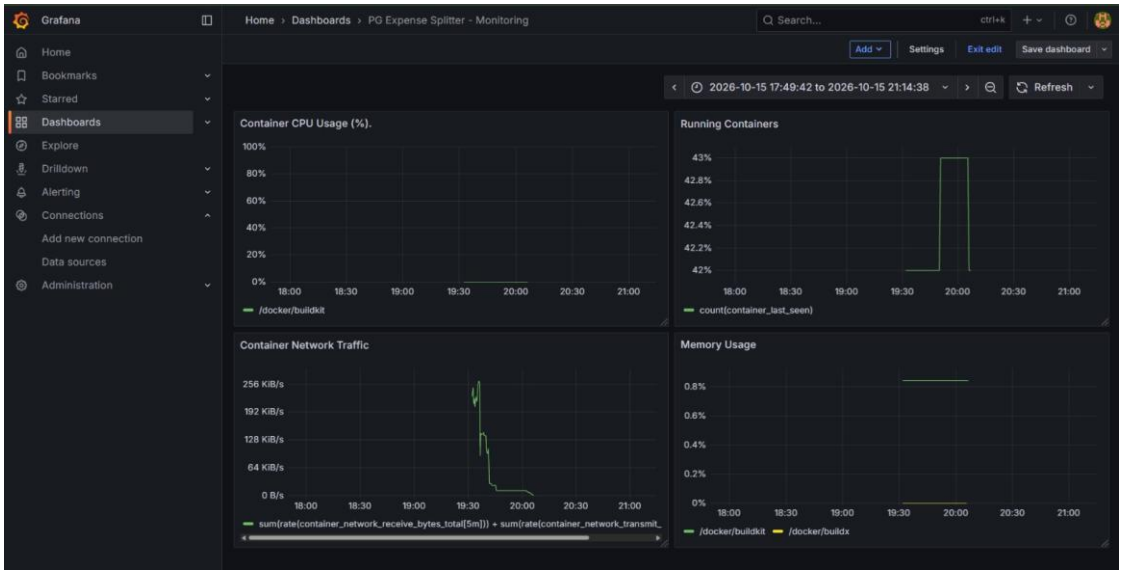


Figure 9.1: Grafana Monitoring Dashboard

9.4 Monitoring Workflow

The monitoring workflow follows these steps:

1. Configure the required monitoring targets.
2. Start the Prometheus service.
3. Open the Prometheus web interface.
4. Check the target status and confirm whether the configured targets are reachable.
5. Configure Grafana to use the appropriate data source.
6. Open the Grafana dashboard to view the available metrics.

The exact metrics displayed depend on the targets and exporters configured in the environment.

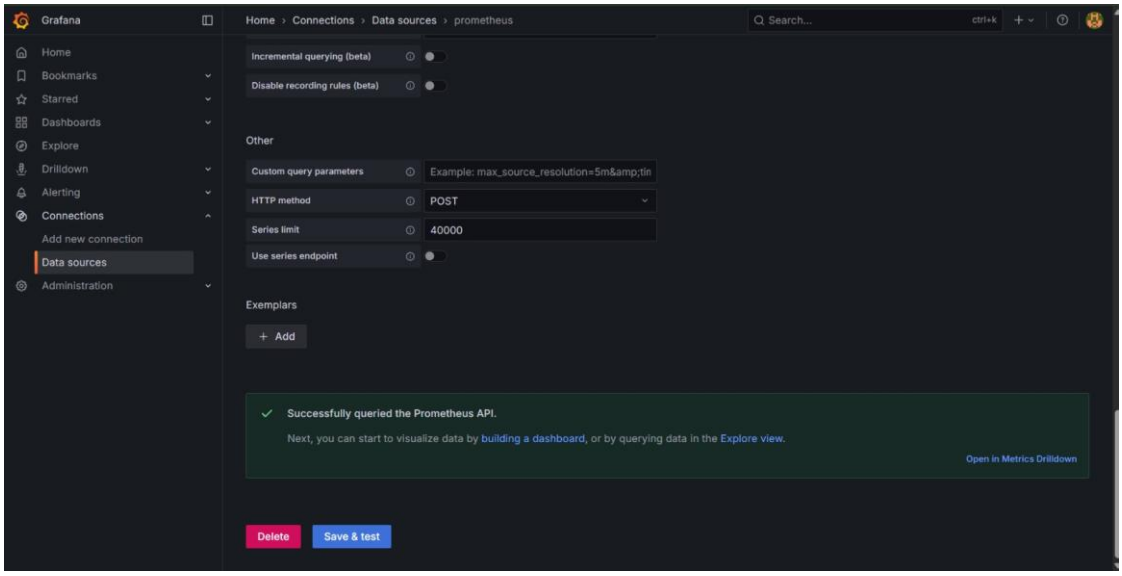


Figure 9.2: Monitoring Configuration Interface

9.5 Benefits of Monitoring

- Helps verify the status of configured targets.
- Provides a visual representation of available metrics.
- Supports observation of system resource usage when the relevant metrics are configured.
- Helps identify monitoring or connectivity issues.
- Improves visibility into the DevOps environment.
- Provides a foundation for future alerting and performance analysis.

9.6 Summary

Prometheus and Grafana provide monitoring and visualisation capabilities for the configured DevOps environment. Prometheus is used to inspect target status and collect metrics, while Grafana presents available data through dashboards. These tools improve system visibility and support future monitoring enhancements for the PG Expense Splitter project.

Chapter 10: Advantages, Applications, and Limitations

10.1 Advantages

1. Easy expense tracking
2. Reduces manual errors
3. Clear payment status visibility
4. Role-based access (Admin/User)
5. User-friendly interface

10.2 Applications

1. PG accommodations
2. Shared flats
3. Hostel expense management
4. Friends traveling together

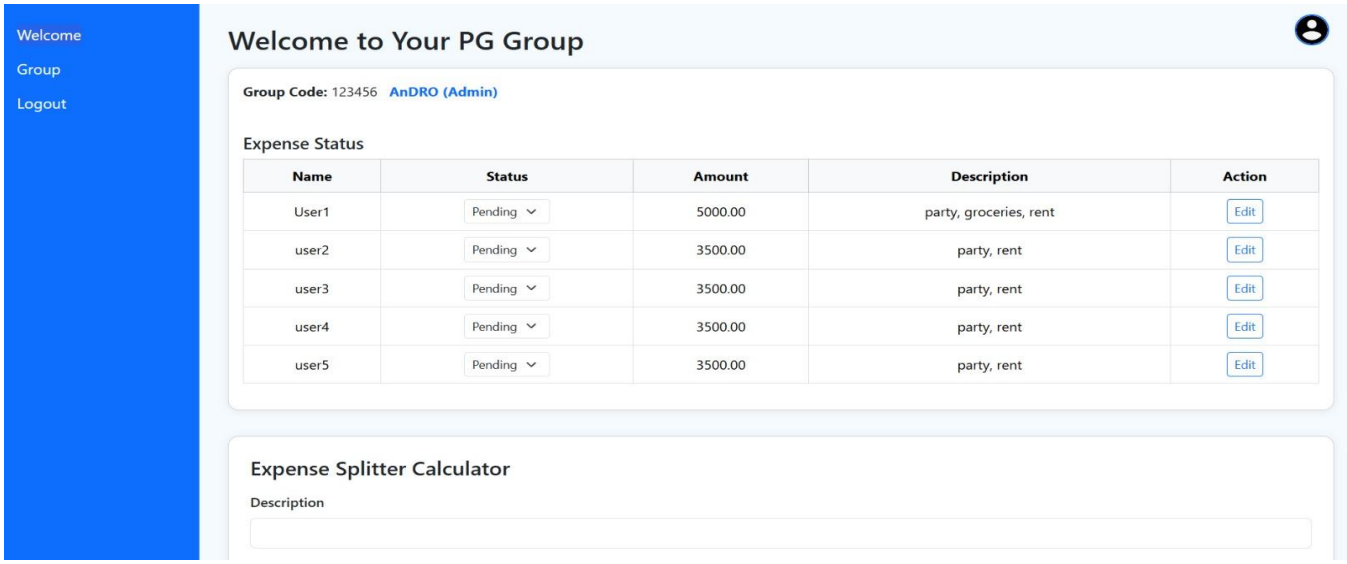


Figure 10.1: PG Expense Splitter Application Interface

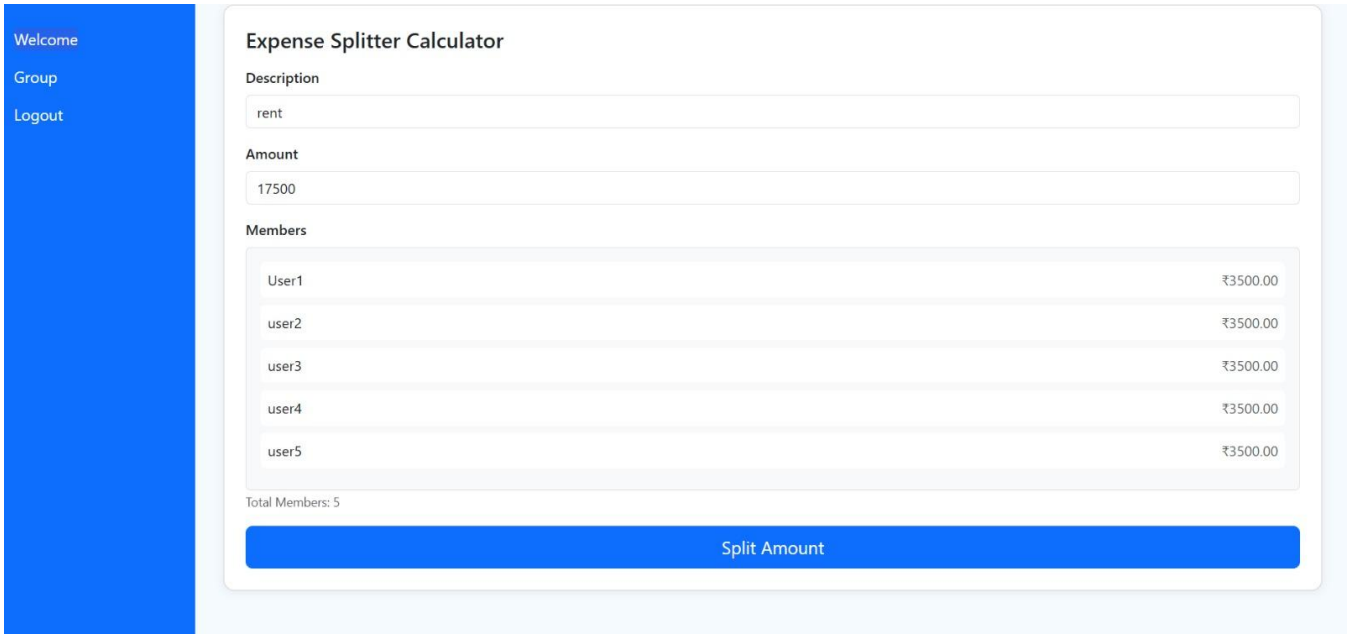


Figure 10.2: Expense Management and Tracking

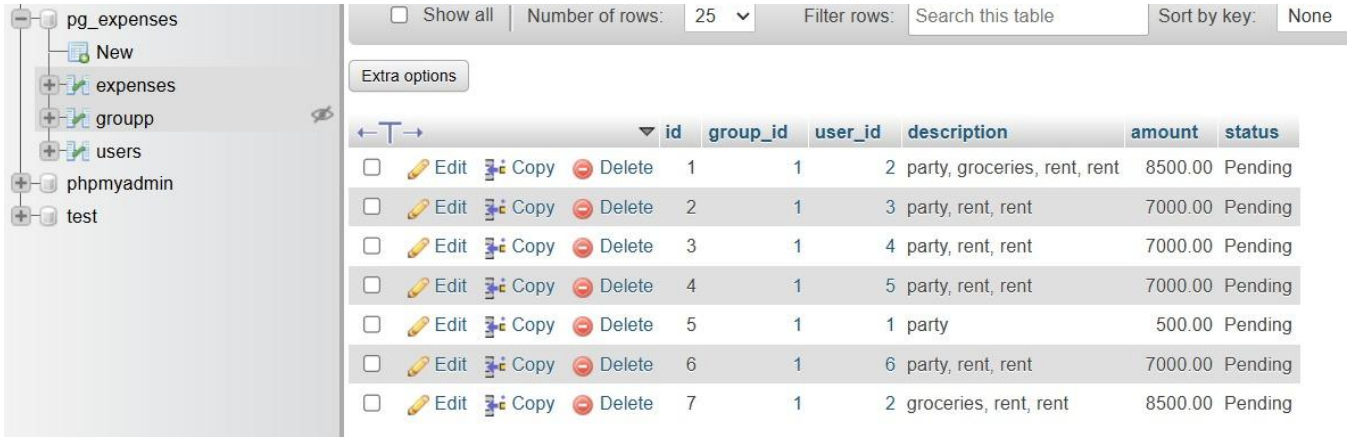


Figure 10.1: Expense Records and Payment Status in the MySQL Database

10.3 Limitations

1. Only admin can manage expenses
2. No payment gateway integration
3. Limited to small groups
4. No real-time notifications

Chapter 11: Conclusion And Future Scope

10.1 Conclusion

The PG Expense Splitter project provides a web-based solution for managing shared expenses in PG accommodations, hostels, and shared living spaces. It helps users organise group expenses, maintain expense records, and improve the visibility of payment-related information.

The project demonstrates the use of HTML, CSS, JavaScript, PHP, and MySQL for application development. DevOps tools such as Git, GitHub, GitHub Actions, Docker, and Docker Compose support source-code management, workflow automation, and containerized execution. Prometheus and Grafana provide monitoring and visualisation capabilities for the configured environment.

Overall, the project combines web application development with DevOps practices and provides a foundation for improving software management, deployment, and monitoring.

10.2 Future Scope

The project can be improved in the following ways:

1. **Online Payment Integration:** Integrate a secure payment gateway to support online expense settlement.
2. **Real-Time Notifications:** Notify users when expenses are added or payments are updated.
3. **Advanced Reports:** Generate monthly expense summaries and downloadable reports.
4. **Mobile Application:** Develop a mobile application for easier access.
5. **Improved Scalability:** Optimise the application to support larger groups.
6. **Automated Deployment:** Extend the CI workflow to include automated deployment.
7. **Enhanced Monitoring:** Configure additional metrics and alerts to monitor system health.

10.3 Final Summary

The PG Expense Splitter project demonstrates how a shared-expense management application can be developed and supported using suitable DevOps tools. The implementation provides a base for future enhancements in payment processing, automation, scalability, and monitoring.

Chapter 12: References

1. **Git Documentation**
   https://git-scm.com/doc
2. **GitHub Documentation**
   https://github.com/Overpowered21/pg-expense-splitter-devops
3. **GitHub Actions Documentation**
   https://docs.github.com/en/actions
   <Cite refs={["turn493189search1","turn493189search4"]}/>
4. **Docker Documentation**
   https://docs.docker.com/get-started/
5. **Docker Compose Documentation**
   https://docs.docker.com/compose/
6. **Prometheus Documentation**
   https://prometheus.io/docs/introduction/overview/
   <Cite refs={["turn493189search0"]}/>
7. **Grafana Documentation**
   https://grafana.com/docs/
   <Cite refs={["turn493189search3"]}/>
8. **PHP Documentation**
   https://www.php.net/docs.php
9. **MySQL Documentation**
   https://dev.mysql.com/doc/
10. **MDN Web Docs – HTML, CSS and JavaScript**
    https://developer.mozilla.org/en-US/docs/Web
11. **Apache Friends – XAMPP**
    https://www.apachefriends.org/
12. **Jenkins Documentation**
    https://www.jenkins.io/doc/
13. **Project Source Code and Implementation Evidence**
    The PG Expense Splitter source code, repository configuration, workflow files, and implementation screenshots used for this project.
14. **Live Deployment Link: PG Expense Splitter**

    https://pg-expense-splitter-devops.onrender.com

    The PG Expense Splitter application is deployed on Render and can be accessed through the above URL when the service is available.